

Assignment 6 – Retrieval Augmented Generation

Introduction

This project implements an end-to-end Retrieval-Augmented Generation (RAG) application using the book *Introduction to Machine Learning with Python* (PDF format). The system retrieves relevant content from the book and generates accurate answers to technical questions related to Machine Learning and Deep Learning.

Data Loading & Preprocessing

The PDF textbook was loaded using PyPDFLoader, which extracted all 392 pages. Preprocessing removed chapter headings, page numbers, and fixed broken words to ensure clean text

Output:

- Total pages: 392
- Sample content displayed from page 1.

Text Chunking

The text was split into chunks of 1500 tokens with 200 overlap using a recursive splitter. This produced 639 chunks, balancing retrieval accuracy with manageable context size.

Output:

- Total chunks: 639

Embeddings & Vector Database

Embeddings were generated using **Google Gemini Embeddings** (gemini-embedding-001), chosen for their semantic richness and seamless integration with LangChain. FAISS was used as the vector database for efficient similarity search.

Why batch processing?

Embeddings were processed in batches of 10 to avoid API quota issues and rate limits. This ensured reliability and complete coverage of all chunks without interruption.

At the same time, Hugging Face models (such as all-MiniLM-L6-v2 or multi-qa-MiniLM-L6-v2) can also be used.

Output:

- Embeddings created successfully
- FAISS index built and populated

Retrieval Pipeline

Queries are converted into embeddings and matched against stored chunks. The system retrieves the top-k relevant passages, with k=5 as the default. This ensures that answers are grounded in the most relevant sections of the textbook.

- Query: “What is overfitting?”

```
answer, docs = rag_query("What is overfitting?")  
  
print("ANSWER:\n", answer)
```

ANSWER:
Overfitting occurs when a model is fit too closely to the particularities of the training set, resulting in a model that works well on the training set but is not able to generalize to new data.

Answer Generation (RAG)

The retrieved context is passed to **Google Gemini 2.5 Flash**, selected for its speed, accuracy, and reduced hallucination. A strict prompt design enforces use of only retrieved content and requires explicit rejection when the answer is not found.

Why this model?

Gemini 2.5 Flash was chosen because it balances efficiency with factual reliability, making it suitable for real-time interactive applications.

```
=====
RAG SYSTEM (Ask Questions from PDF)
Type 'exit' to stop the application
=====

-----
Ask Question: what is unsupervised learning?

🔍 Searching relevant context...

💡 ANSWER:
-----
Unsupervised learning subsumes all kinds of machine learning where there is no known output, no teacher to instruct the learning algorithm. In unsupervised learning, the learning algorithm is just shown the input data and asked to extract knowledge from this data.
-----

-----
Ask Question: differentiate regression and classification?

🔍 Searching relevant context...

💡 ANSWER:
-----
There are two major types of supervised machine learning problems: classification and regression.

* **Classification:** The goal is to predict a class label, which is a choice from a predefined list of possibilities. Classification tasks can be binary (distinguishing between exactly two classes, like a yes/no question, e.g., spam or not spam) or multiclass (classification between more than two classes, e.g., classifying iris flowers into three species). There is no continuity between possible outcomes in classification.
* **Regression:** This type of problem involves predicting an outcome where there is some kind of continuity in the output. For example, predicting annual income, where small differences in the predicted amount do not make a tangible difference.

The key difference is whether there is continuity in the output: if there is continuity between possible outcomes, it's a regression problem; if there is no continuity (discrete categories), it's a classification problem.
-----
```

All components were integrated into an interactive command-line RAG system. Users can ask questions, receive grounded answers, and exit gracefully. Demonstrations

show correct answers for in-scope queries and proper rejection for out-of-scope queries. **Output:**

- Query: “*Explain Django framework?*” → Correct rejection.
- Query: “*Exit*” → Application closed gracefully.

Ask Question: what is the capital of india?

 Searching relevant context...

 ANSWER:

The provided context does not mention the answer to this question.

Ask Question: explain django framework?

 Searching relevant context...

 ANSWER:

The provided context does not mention the answer to this question.

Ask Question: exit

Exiting RAG system!
